Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №4
на тему
«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ. КРИПТОСИСТЕМА МАК-ЭЛИСА»
БГУИР 6-05-0612-02 21

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ПОЛЯКОВ Кирилл Борисович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Краткие теоретические сведения 4
2.1 Код Гоппы 4
2.2 Маскировка ключа 4
2.3 Стойкость и параметры 4
3 Выполнение работы 6
3.1 Структура программного средства 6
3.2 Генерация ключей 6
3.3 Зашифрование 7
3.4 Расшифрование по Паттерсону 7
3.5 Кадрирование и размеры ключей 8
3.6 Интерфейс 8
3.7 Тестирование 9
Заключение 10
Список использованных источников 11
Приложение А (Обязательное) Листинг программного кода 12

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы является изучение асимметричной криптосистемы Мак-Элиса и получение практических навыков построения криптосистем на основе кодов, исправляющих ошибки [1].

В ходе работы требовалось реализовать программное средство шифрования и дешифрования текстовых файлов криптосистемой Мак-Элиса для криптостойких параметров порождающей матрицы. Для этого необходимо:

1 построить бинарный код Гоппы над полем $GF(2^m)$ по неприводимому многочлену степени t;

2 реализовать генерацию ключей с маскировкой порождающей матрицы случайной обратимой матрицей и перестановкой координат;

3 реализовать зашифрование с добавлением случайного вектора ошибок;

4 реализовать расшифрование алгоритмом Паттерсона [2];

5 задать кадрирование сообщения и двоичный формат ключей;

6 встроить криптосистему в терминальный интерфейс программы и проверить реализацию тестами.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Код Гоппы

Бинарный код Гоппы [3] задаётся многочленом $g(z)$ степени t над полем $\mathrm{GF}(2^m)$ и упорядоченным набором опорных элементов $a_0, a_1, \ldots, a_{n-1}$ — всех элементов поля, так что $n = 2^m$. Кодовое слово $c = (c_0, c_1, \ldots, c_{n-1})$ принадлежит коду, если

$$\sum_{i=0}^{n-1} \frac{c_i}{z - a_i} \equiv 0 \pmod{g(z)}. \tag{2.1}$$

Проверочная матрица H размера $mt \times n$ устроена так: столбец i — это коэффициенты разложения $\frac{1}{z-a_i} \bmod g(z)$ по базису $1, z, \ldots, z^{t-1}$, записанные m -битными элементами поля. Для неприводимого g размерность кода $k \geq n - mt$, а минимальное расстояние не меньше $2t + 1$, поэтому код исправляет до t ошибок.

2.2 Маскировка ключа

Порождающая матрица кода Гоппы имеет регулярную структуру, и по ней легко восстановить секретный многочлен. Поэтому в схеме Мак-Элиса открытым ключом служит матрица

$$G_1 = S \cdot G \cdot P, \tag{2.2}$$

где G — порождающая матрица кода Гоппы, S — случайная обратимая $k \times k$ матрица над $\mathrm{GF}(2)$, P — случайная $n \times n$ матрица перестановки. Произведение SGP выглядит как порождающая матрица произвольного линейного кода: скрыты и систематический вид G, и порядок опорных элементов.

2.3 Стойкость и параметры

Стойкость схемы опирается на трудность декодирования произвольного линейного кода: для кода общего вида поиск слова веса не больше t в окрестности принятого вектора считается трудной задачей, тогда как для кода Гоппы с известным g тот же поиск выполняет полиномиальный алгоритм Паттерсона. Криптосистема предложена в 1978 году и переживает возвращение интереса в постквантовой криптографии: атаки на неё не используют факторизацию или дискретный логарифм.

Параметры из методических указаний [4] — $n = 1024$, $m = 10$, $t = 38$, $k = 644$ — дают учебный уровень стойкости: оценки атак информационного множества для таких коротких кодов заметно ниже 128 бит. Практические параметры требуют $n \geq 4096$ и вместе с ними открытого ключа в сотни килобайт.

Именно размер ключа (около 82 КБ против сотен байт у RSA) и медленная генерация ключей стали причиной того, что схема не получила широкого распространения [5].

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программного средства

Криптосистема реализована на языке Go без внешних зависимостей и разделена на уровни:

– *crypto/mceliece/gf.go* — арифметика поля $\mathrm{GF}(2^m)$ на таблицах логарифмов;

– *crypto/mceliece/poly.go* — многочлены над полем: деление, НОД, расширенный Евклид, обратный элемент, корень по модулю;

– *crypto/mceliece/matrix.go* — двоичные матрицы и битовые векторы;

– *crypto/mceliece/goppa.go* — построение кода Гоппы, проверка неприводимости, приведение проверочной матрицы к систематическому виду;

– *crypto/mceliece/patterson.go* — декодер Паттерсона;

– *crypto/mceliece/mceliece.go* — генерация ключей, зашифрование, расшифрование, кадрирование;

– *crypto/mceliece/keys.go* — двоичный формат открытого и закрытого ключей;

– *cipher/mceliece.go* — адаптер к интерфейсу *AsymCipher*

– *ui/asympane* — существующая панель асимметричного шифрования.

Новая панель не создавалась: криптосистема подключена к уже готовой вкладке *asympane*, как и требовалось в задании.

3.2 Генерация ключей

Порядок генерации показан на рисунке 3.1. Случайный приведённый многочлен $g(z)$ степени t проверяется критерием Рабина: для всех $i = 1, 2, \ldots, \frac{t}{2}$ должно выполняться $\gcd\left(g(z), z^{2^{mi}} - z\right) = 1$, а на последнем шаге — $z^{2^{mt}} \equiv z \pmod{g}$. Генерация повторяется, пока многочлен не окажется неприводимым.

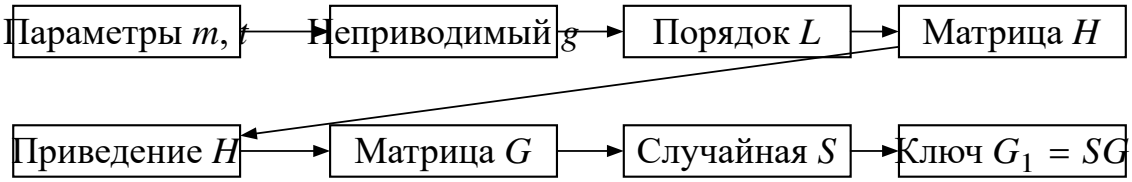


Рисунок 3.1 – Схема генерации ключей Мак-Элиса

Перестановка координат P объединена со случайным порядком опорных элементов: при приведении H к виду $[A \mid I]$ столбцы меняются местами, и вместе с ними переставляется список L. Открытым ключом является пара (G_1, t), закрытым — многочлен g, список L и матрица S^{-1}. Случайная матрица S принимается только после успешного обращения (метод Гаусса — Жордана); вероятность обратимости случайной двоичной матрицы близка к 0.29, поэтому в среднем нужно около четырёх попыток.

3.3 Зашифрование

Сообщение переводится в битовый поток с 32-битной длиной в начале, делится на блоки по k бит, последний блок дополняется нулями. Каждый блок шифруется по формуле

$$C = M \cdot G_1 + Z, \tag{3.1}$$

где M — k-битный блок сообщения, а Z — случайный вектор ошибок веса ровно t (рисунок 3.2). Случайный вектор выбирается частичным перемешиванием Фишера — Йетса, поэтому позиции ошибок не повторяются. Шифртекст — конкатенация блоков по n бит, то есть $\frac{n}{8}$ байт на блок.

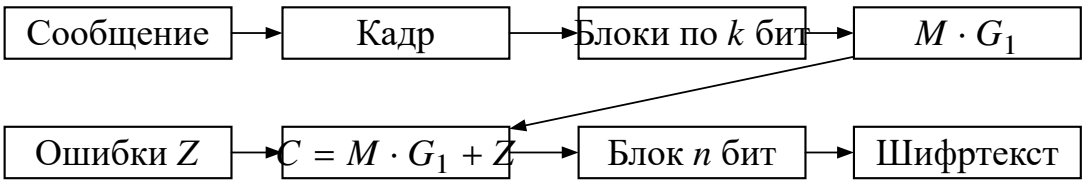


Рисунок 3.2 – Схема зашифрования блока сообщения

3.4 Расшифрование по Паттерсону

Получатель работает с секретными g и L. Сначала вычисляется синдром принятого слова r:

$$S(z) = \sum_{i:r_i=1} \frac{1}{z - a_i} \equiv 0 \pmod{g(z)}. \tag{3.2}$$

Если $S(z) = 0$, ошибок нет. Иначе строится

$$T(z) = \sqrt{z + S^{-1}(z)} \pmod{g(z)}, \tag{3.3}$$

а затем расширенный алгоритм Евклида для пары (g, T) останавливается на первом остатке степени не выше $\frac{t}{2}$. Полученные $a(z)$ и $b(z)$ дают многочлен локаторов ошибок

$$\sigma(z) = a(z)^2 + z \cdot b(z)^2. \tag{3.4}$$

Его корни среди опорных элементов L и есть позиции ошибок. Если число найденных корней не совпадает со степенью σ, расшифрование считается неудачным: ошибок больше t. Затем исправленное слово даёт M_1 (первые k бит), и сообщение восстанавливается как $M = M_1 \cdot S^{-1}$. Последовательность показана на рисунке 3.3.

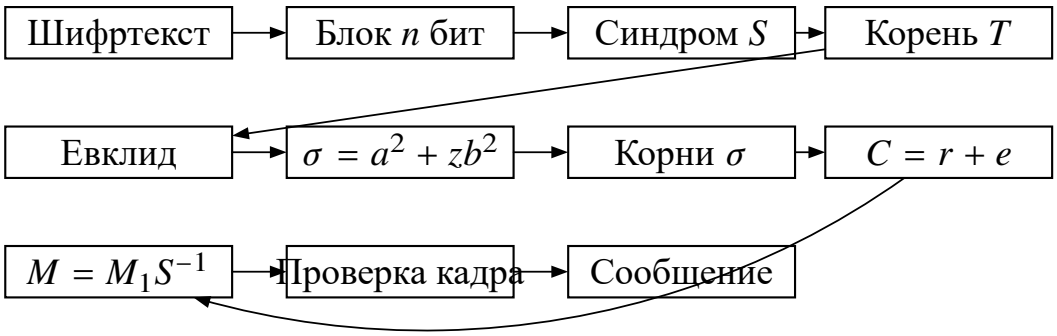


Рисунок 3.3 – Схема расшифрования алгоритмом Паттерсона

Квадратный корень в шаге $T(z)$ вычисляется в фактор-кольце $\mathrm{GF}(2^m)\frac{[z]}{g(z)}$. Так как g неприводим, это поле $\mathrm{GF}(2^{mt})$, в котором возведение в квадрат — биекция, а обратная операция равна $x^{2^{mt-1}}$, то есть $mt - 1$ последовательному возведению в квадрат по модулю g. Такой способ корректен для любого аргумента, тогда как формула «корень из чётных степеней» применима только к точным квадратам.

3.5 Кадрирование и размеры ключей

Кадр — это битовый поток: 32-битная длина сообщения в битах, записанная старшим битом вперёд, затем биты сообщения. Поток делится на блоки по k бит, последний блок дополняется нулями; пустое сообщение всё равно даёт один блок, потому что кадр содержит длину. При расшифровании проверяются: кратность длины байтам, соответствие числа блоков длине и нулевое дополнение последнего блока. Собственной проверки целостности у схемы Мак-Элиса нет, поэтому эти проверки лишь отбрасывают большинство случайных результатов при неверном ключе, но не заменяют имитовставку.

Размеры для двух наборов параметров приведены в таблице 3.1. Расширение шифртекста равно $\frac{n}{k}$: каждый блок сообщения длиной k бит превращается в блок длиной n бит.

Таблица 3.1 – Параметры и размеры ключей

Параметр	$n = 1024, t = 38$	$n = 256, t = 8$
Размерность k	644	192
Открытый ключ	82 452 Б	6 164 Б
Закрытый ключ	53 992 Б	5 162 Б
Расширение $\frac{n}{k}$	1.59	1.33

3.6 Интерфейс

Криптосистема встроена в существующую вкладку *McEliece* панели *asympane*. Клавиша *Ctrl+G* генерирует ключевую пару выбранного набора параметров (переключение — *Enter* на строке параметров), закрытый ключ скрыт и показывается по *Ctrl+R*.

Открытый ключ Мак-Элиса занимает десятки килобайт, поэтому в поле ключа он не помещается. Панель хранит такой ключ как последовательность байтов и показывает в поле компактную подпись вида `<binary key: 82452 B>` зашифрование и расшифрование используют сохранённые байты. Клавиша *Ctrl+E* выгружает оба ключа в файлы *mceliece-public.key* и *mceliece-private.key* в шестнадцатеричном виде, *Ctrl+L* загружает их обратно с проверкой формата. Остальные возможности панели — ввод вручную или из файла, переключение режимов *Encrypt*/*Decrypt*, сохранение результата — не изменились.

3.7 Тестирование

Корректность проверена модульными тестами пакета *crypto/mceliece* командой `go test ./...`. Все тесты детерминированы: генератор случайности заменяется воспроизводимым читателем, поэтому результаты не зависят от запуска.

– Арифметика поля проверяется на обратном элементе, квадратном корне, дистрибутивности для обоих полей $\mathrm{GF}(2^8)$ и $\mathrm{GF}(2^{10})$.

– Для кода проверяются: ранг порождающей матрицы k, систематический вид $[I_k \mid A^T]$, равенство $H \cdot G^T = 0$, неприводимость g и то, что опорные элементы образуют перестановку.

– Отдельный тест восстанавливает $S = \left(S^{-1}\right)^{-1}$ из закрытого ключа и проверяет $G_1 = S \cdot G$.

– Декодер возвращает исходное кодовое слово при t ошибках для обоих наборов параметров, не меняет слово без ошибок и не выдаёт исходное слово при $t + 1$ ошибке.

– Круговое шифрование и расшифрование проверено для пустого сообщения, одного байта, сообщения из нескольких блоков и 1 КБ случайных двоичных данных; длина шифртекста сверяется с формулой $\left\lceil \frac{32+8\,|M|}{k} \right\rceil \cdot \frac{n}{8}$.

– Негативные тесты: обрезанный и пустой шифртекст, лишний блок и чужой закрытый ключ дают ошибку; испорченная сигнатура или обрезанный ключ отвергаются при разборе; дубликат опорного элемента в закрытом ключе отвергается.

– Сериализация ключей проверена на побайтовое совпадение после разбора и обратной записи.

Дополнительно вкладка проверена в терминале: генерация ключей, зашифрование текста, перенос шифртекста в поле ввода, переключение в режим расшифрования и восстановление исходного текста.

ЗАКЛЮЧЕНИЕ

В ходе работы реализована криптосистема Мак-Элиса на бинарных кодах Гоппы: построение кода по неприводимому многочлену, маскировка порождающей матрицы $S \cdot G$ со случайным порядком опорных элементов, зашифрование $C = M \cdot G_1 + Z$ с вектором ошибок веса t и расшифрование алгоритмом Паттерсона. Сообщения кадрируются 32-битной длиной и шифруются блоками по k бит; ключи хранятся в собственном двоичном формате с проверкой размеров при разборе.

Криптосистема подключена к существующей панели асимметричного шифрования; для больших ключей панель показывает подпись и работает с сохранёнными байтами, а *Ctrl+E* и *Ctrl+L* позволяют выгрузить и загрузить ключи. Для основного набора $n = 1024$, $t = 38$, $k = 644$ открытый ключ занимает около 82 КБ, закрытый — около 54 КБ, а шифртекст в $\frac{n}{k} \approx 1.59$ раза длиннее открытого текста.

Ограничением реализации является отсутствие собственной проверки целостности: схема Мак-Элиса не аутентифицирует сообщение, поэтому неверный ключ или повреждение шифртекста обнаруживаются только по проверкам кадра и синдрома. Кроме того, набор параметров из методических указаний является учебным и не обеспечивает современный уровень стойкости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] McEliece, R.J. A Public-Key Cryptosystem Based on Algebraic Coding Theory / R.J. McEliece. – 1978.

[2] Patterson, N.J. The Algebraic Decoding of Goppa Codes / N.J. Patterson // IEEE Transactions on Information Theory. – 1975. – Т. 21, № 2. – С. 203-207.

[3] Goppa, V.D. A New Class of Linear Error-Correcting Codes / V.D. Goppa // Problems of Information Transmission. – 1970. – Т. 6, № 3. – С. 24-30.

[4] Лабораторные работы по дисциплине «Методы защиты информации» : методические указания. – 2024.

[5] Bernstein, D.J. Attacking and Defending the McEliece Cryptosystem / D.J. Bernstein, T. Lange, C. Peters // Post-Quantum Cryptography. – Springer, 2008. – С. 31-46.

[6] Menezes, A.J. Handbook of Applied Cryptography / A.J. Menezes, P.C. van Oorschot, S.A. Vanstone. – Boca Raton: CRC Press, 1996. – 780 с.

ПРИЛОЖЕНИЕ А
(Обязательное)
Листинг программного кода

Листинг А.1 – Арифметика поля GF(2^m)

```
package mceliece

import "fmt"

// Арифметика поля GF(2^m).
//
// Элементы поля — m-битные слова uint16. Сложение в характеристике 2
// совпадает с побитовым XOR, поэтому вычитание не отличается от сложения.
// Умножение реализовано через таблицы логарифмов и антилогарифмов:
// ненулевые элементы поля — это степени образующего элемента z, и
// a*b = z^(log a + log b).

// fieldPolynomials задаёт представление GF(2^m): примитивный многочлен,
// по которому выполняется приведение при умножении.
var fieldPolynomials = map[int]uint16{
  8:  0x11D, // z^8 + z^4 + z^3 + z^2 + 1
  10: 0x409, // z^10 + z^3 + 1
}

// Field — поле GF(2^m) с готовыми таблицами логарифмов.
type Field struct {
  m    int
  size int    // 2^m
  poly uint16 // примитивный многочлен поля
  log  []int  // log[x] — степень z, дающая x; log[0] не используется
  exp  []uint16
}

// NewField строит таблицы поля GF(2^m). Поддерживаются m = 8 и m = 10.
func NewField(m int) (*Field, error) {
  poly, ok := fieldPolynomials[m]
  if !ok {
    return nil, fmt.Errorf("mceliece: no field polynomial for GF(2^%d)", m)
  }

  size := 1 << m
  f := &Field{
    m:    m,
    size: size,
    poly: poly,
    log:  make([]int, size),
    // Таблица удвоена: сумма логарифмов лежит в [0, 2*(size-1)),
    // поэтому брать её по модулю при умножении не нужно.
    exp: make([]uint16, 2*(size-1)),
  }

  // Образующий элемент — сам z; если многочлен примитивен, его степени
  // обходят все ненулевые элементы поля ровно один раз. Таблица
  // заполняется дважды: вторая половина — те же степени, сдвинутые на
  // period = size-1, чтобы индексы сумм логарифмов не брать по модулю.
  x := uint16(1)
  period := size - 1
  for i := 0; i < period; i++ {
    if i > 0 && x == 1 {
     return nil, fmt.Errorf("mceliece: polynomial 0x%X is not primitive", poly)
    }
```

```
    f.exp[i] = x
    f.exp[i+period] = x
    f.log[x] = i
    x = f.mulRaw(x, 2)
  }
  if x != 1 {
    return nil, fmt.Errorf("mceliece: polynomial 0x%X is not primitive", poly)
  }
  return f, nil
}

func (f *Field) Degree() int { return f.m }

func (f *Field) Size() int { return f.size }

// mulRaw умножает два элемента без таблиц: сдвиги и приведение по
// многочлену поля. Нужен только при построении таблиц.
func (f *Field) mulRaw(a, b uint16) uint16 {
  var result uint16
  for b != 0 {
    if b&1 != 0 {
      result ^= a
    }
    b >>= 1
    a <<= 1
    if a&uint16(f.size) != 0 {
      a ^= f.poly
    }
  }
  return result
}

// Add — сложение в GF(2^m), оно же вычитание.
func (f *Field) Add(a, b uint16) uint16 { return a ^ b }

// Mul умножает через таблицы логарифмов.
func (f *Field) Mul(a, b uint16) uint16 {
  if a == 0 || b == 0 {
    return 0
  }
  return f.exp[f.log[a]+f.log[b]]
}

// Square возводит элемент в квадрат.
func (f *Field) Square(a uint16) uint16 {
  if a == 0 {
    return 0
  }
  return f.exp[2*f.log[a]]
}

// Inv возвращает обратный элемент; Inv(0) = 0, вызывающий код не должен
// передавать ноль.
func (f *Field) Inv(a uint16) uint16 {
  if a == 0 {
    return 0
  }
  return f.exp[f.size-1-f.log[a]]
}

// Div делит a на b через таблицы логарифмов.
func (f *Field) Div(a, b uint16) uint16 {
  if a == 0 || b == 0 {
```

```
    return 0
  }
  return f.exp[f.log[a]-f.log[b]+f.size-1]
}

// Sqrt вычисляет квадратный корень: в характеристике 2 возведение в
// квадрат — биекция (x^(2^m) = x), поэтому sqrt(x) = x^(2^(m-1)), а в
// логарифмах это деление показателя на 2 по модулю size-1.
func (f *Field) Sqrt(a uint16) uint16 {
  if a == 0 {
    return 0
  }
  n := f.size - 1
  k := f.log[a]
  if k%2 != 0 {
    k += n // n нечётно, поэтому k станет чётным
  }
  return f.exp[k/2]
}

// Elements возвращает все элементы поля в фиксированном порядке 0..2^m-1.
func (f *Field) Elements() []uint16 {
  all := make([]uint16, f.size)
  for i := range all {
    all[i] = uint16(i)
  }
  return all
}
```

Листинг А.2 – Декодер Паттерсона

```
package mceliece

// Декодирование бинарного кода Гоппы алгоритмом Паттерсона.
//
// Вход — принятое слово r = c + e, где c — кодовое слово, а e содержит не
// более t ошибок. Синдром S(z) = Σ e_i/(z - a_i) mod g равен нулю тогда и
// только тогда, когда ошибок нет. Далее по S строится многочлен
// локаторов ошибок σ(z) = a(z)^2 + z*b(z)^2, корни которого — опорные
// элементы с ошибками.
//
// Примечание о квадратном корне: z + S^{-1} не раскладывается на чётные
// степени z напрямую, поэтому корень извлекается в фактор-кольце
// GF(2^m)[z]/g: для неприводимого g это поле GF(2^(m*t)), в котором
// возведение в квадрат — биекция с обратной x^(2^(m*t-1)).

// Decoder — декодер Паттерсона для кода Гоппы, заданного многочленом g и
// упорядоченным списком опорных элементов.
type Decoder struct {
  Field   *Field
  Params  Params
  G       Poly
  Support []uint16
}

// Decode исправляет до t ошибок в принятом слове и возвращает кодовое слово.
// Если ошибок больше t или синдром не согласован, возвращается ErrDecode.
func (d *Decoder) Decode(received []uint64) ([]uint64, error) {
  if len(d.Support) != d.Params.N || len(received)*64 < d.Params.N {
    return nil, ErrDecode
  }

  syndrome, err := d.syndrome(received)
```

```
  if err != nil {
    return nil, err
  }
  if len(syndrome) == 0 {
    return append([]uint64(nil), received...), nil
  }

  // T(z) = sqrt(z + S(z)^{-1}) mod g.
  inverse, err := d.Field.PolyInv(syndrome, d.G)
  if err != nil {
    return nil, err
  }
  target := d.Field.PolyAdd(Poly{0, 1}, inverse)
  t, err := d.Field.PolySqrtMod(target, d.G)
  if err != nil {
    return nil, err
  }

  // Расширенный алгоритм Евклида для (g, T); останавливаемся на первом
  // остатке степени не выше t/2.
  a, b := d.euclidean(t, Deg(d.G)/2)

  // σ(z) = a(z)^2 + z*b(z)^2.
  sigma := d.Field.PolyAdd(d.Field.PolySquare(a), d.Field.PolyMul(Poly{0, 1},
d.Field.PolySquare(b)))

  errors := newVector(d.Params.N)
  found := 0
  for position, element := range d.Support {
    if d.Field.PolyEval(sigma, element) == 0 {
      setVectorBit(errors, position)
      found++
    }
  }
  // У корректного синдрома σ раскладывается на столько линейных
  // множителей, какова его степень; иначе исправление невозможно.
  if found == 0 || found != Deg(sigma) {
    return nil, ErrDecode
  }

  corrected := make([]uint64, len(received))
  for i := range received {
    corrected[i] = received[i] ^ errors[i]
  }
  return corrected, nil
}

// syndrome вычисляет S(z) = Σ_{r_i=1} 1/(z - a_i) mod g.
func (d *Decoder) syndrome(received []uint64) (Poly, error) {
  sum := Poly(nil)
  for position, element := range d.Support {
    if !vectorBit(received, position) {
      continue
    }
    // 1/(z - a) = 1/(z + a) в характеристике 2.
    inverse, err := d.Field.PolyInv(Poly{element, 1}, d.G)
    if err != nil {
      return nil, err
    }
    sum = d.Field.PolyAdd(sum, inverse)
  }
  return sum, nil
}
```

```
// euclidean выполняет расширенный алгоритм Евклида для g и t и возвращает
// пару (a, b), для которой a ≡ b*T (mod g) и deg a, deg b <= t/2.
func (d *Decoder) euclidean(t Poly, half int) (Poly, Poly) {
  rPrev, rCur := d.G, t
  bPrev, bCur := Poly(nil), Poly{1}

  for Deg(rCur) > half {
    q, rNext := d.Field.PolyDivMod(rPrev, rCur)
    bNext := d.Field.PolyAdd(bPrev, d.Field.PolyMul(q, bCur))
    rPrev, rCur = rCur, rNext
    bPrev, bCur = bCur, bNext
  }
  return rCur, bCur
}
```

Листинг А.3 – Криптосистема Мак-Элиса

```
package mceliece

import (
  "crypto/rand"
  "fmt"
  "io"
)

// Криптосистема Мак-Элиса.
//
// Открытый ключ — порождающая матрица G1 = S*G*P, где G — систематическая
// порождающая матрица кода Гоппы, S — случайная обратимая k×k матрица,
// P — перестановка координат. В реализации P объединена со случайным
// порядком опорных элементов, поэтому хранится не матрица перестановки,
// а уже переставленный список опорных элементов support.
//
// Зашифрование: C = M*G1 + Z, где M — k-битный блок сообщения, Z — вектор
// ошибок веса ровно t.
//
// Расшифрование: принятое слово декодируется Паттерсоном с секретными g и
// support, что даёт M*S, затем умножается на S^{-1}.

const (
  publicMagic  = "MZIMCEPK"
  privateMagic = "MZIMCESK"
  keyVersion   = 1

  publicHeaderSize  = 20
  privateHeaderSize = 24
  framingBits       = 32
)

// PublicKey — открытый ключ Мак-Элиса.
type PublicKey struct {
  Params Params
  Matrix *Matrix // G1, k × n
}

// PrivateKey — закрытый ключ Мак-Элиса.
type PrivateKey struct {
  Params   Params
  G        Poly
  Support  []uint16
  SInverse *Matrix // S^{-1}, k × k
}
```

```
// GenerateKeyPair создаёт ключевую пару. random — источник случайности;
// в рабочем коде это crypto/rand, в тестах — детерминированный читатель.
func GenerateKeyPair(params Params, random io.Reader) (*PublicKey, *PrivateKey,
error) {
  code, err := NewGoppaCode(params, random)
  if err != nil {
    return nil, nil, err
  }

  scrambler, scramblerInverse, err := randomInvertibleMatrix(params.K, random)
  if err != nil {
    return nil, nil, err
  }
  // G1 = S*G: строки G складываются по строкам S.
  public, err := scrambler.Mul(code.Matrix)
  if err != nil {
    return nil, nil, err
  }

  return &PublicKey{Params: params, Matrix: public},
     &PrivateKey{Params: params, G: code.G, Support: code.Support, SInverse:
scramblerInverse},
    nil
}

// GenerateKeys создаёт ключевую пару и возвращает её в двоичном формате.
func GenerateKeys(params Params) (public, private []byte, err error) {
  publicKey, privateKey, err := GenerateKeyPair(params, rand.Reader)
  if err != nil {
    return nil, nil, err
  }
  return publicKey.MarshalBinary(), privateKey.MarshalBinary(), nil
}

// Encrypt зашифровывает сообщение случайным вектором ошибок.
func (k *PublicKey) Encrypt(message []byte) ([]byte, error) {
  return k.EncryptWithRandom(message, rand.Reader)
}

// EncryptWithRandom — зашифрование с явным источником случайности.
func (k *PublicKey) EncryptWithRandom(message []byte, random io.Reader) ([]byte,
error) {
  params := k.Params
  if k.Matrix == nil || k.Matrix.Rows != params.K || k.Matrix.Cols != params.N {
    return nil, ErrBadPublicKey
  }
  if len(message) > (1<<32-1)/8 {
    return nil, fmt.Errorf("mceliece: message is longer than 512 MiB")
  }

  framed, totalBits := frameMessage(message)
  // Блоки по k бит; последний дополняется нулями. Пустое сообщение
  // всё равно даёт один блок: кадр содержит 32-битную длину.
  blocks := (totalBits + params.K - 1) / params.K
  blockBytes := params.N / 8
  cipher := make([]byte, blocks*blockBytes)

  for block := 0; block < blocks; block++ {
    messageBlock := newVector(params.K)
    for bit := 0; bit < params.K; bit++ {
      source := block*params.K + bit
      if source < totalBits && vectorBit(framed, source) {
```

```
        setVectorBit(messageBlock, bit)
      }
    }

    codeword := k.Matrix.MulVector(messageBlock)
    errors, err := randomErrorVector(params.N, params.T, random)
    if err != nil {
      return nil, err
    }
    for i := range codeword {
      codeword[i] ^= errors[i]
    }
    copy(cipher[block*blockBytes:], bitsToBytes(codeword, params.N))
  }
  return cipher, nil
}

// Decrypt расшифровывает сообщение закрытым ключом.
func (k *PrivateKey) Decrypt(cipher []byte) ([]byte, error) {
  params := k.Params
  blockBytes := params.N / 8
  if len(cipher) == 0 || len(cipher)%blockBytes != 0 {
    return nil, ErrBadCipher
  }
   if k.SInverse == nil || k.SInverse.Rows != params.K || k.SInverse.Cols !=
params.K {
    return nil, ErrBadPrivateKey
  }
  field, err := NewField(params.M)
  if err != nil {
    return nil, err
  }

  decoder := &Decoder{Field: field, Params: params, G: k.G, Support: k.Support}
  blocks := len(cipher) / blockBytes
  combined := newVector(blocks * params.K)

  for block := 0; block < blocks; block++ {
     received := bitsFromBytes(cipher[block*blockBytes:(block+1)*blockBytes],
params.N)
    corrected, err := decoder.Decode(received)
    if err != nil {
      return nil, fmt.Errorf("%w: block %d: %v", ErrDecode, block, err)
    }
    // Первые k бит кодового слова — это M*S; умножаем на S^{-1}.
    message := k.SInverse.MulVector(corrected)
    for bit := 0; bit < params.K; bit++ {
      if vectorBit(message, bit) {
        setVectorBit(combined, block*params.K+bit)
      }
    }
  }

  return unframeMessage(combined, blocks, params)
}

// frameMessage добавляет к сообщению 32-битную длину в битах. Бит 0 —
// старший бит первого байта сообщения.
func frameMessage(message []byte) ([]uint64, int) {
  totalBits := framingBits + 8*len(message)
  framed := newVector(totalBits)

  length := uint32(8 * len(message))
```

```
  for bit := 0; bit < framingBits; bit++ {
    if length&(1<<uint(framingBits-1-bit)) != 0 {
      setVectorBit(framed, bit)
    }
  }
  for bit := 0; bit < 8*len(message); bit++ {
    if message[bit/8]&(0x80>>uint(bit%8)) != 0 {
      setVectorBit(framed, framingBits+bit)
    }
  }
  return framed, totalBits
}

// unframeMessage проверяет длину и нулевое дополнение последнего блока.
func unframeMessage(bits []uint64, blocks int, params Params) ([]byte, error) {
  if blocks*params.K < framingBits {
    return nil, ErrBadCipher
  }

  var length uint32
  for bit := 0; bit < framingBits; bit++ {
    length = length<<1 | boolToBit(vectorBit(bits, bit))
  }
  if length%8 != 0 || uint64(length) > uint64(blocks*params.K-framingBits) {
    return nil, ErrBadCipher
  }

  totalBits := framingBits + int(length)
  if (totalBits+params.K-1)/params.K != blocks {
    return nil, ErrBadCipher
  }
  for bit := totalBits; bit < blocks*params.K; bit++ {
    if vectorBit(bits, bit) {
      return nil, ErrBadCipher
    }
  }

  message := make([]byte, length/8)
  for i := range message {
    for bit := 0; bit < 8; bit++ {
      if vectorBit(bits, framingBits+i*8+bit) {
        message[i] |= 0x80 >> uint(bit)
      }
    }
  }
  return message, nil
}

func boolToBit(value bool) uint32 {
  if value {
    return 1
  }
  return 0
}

func randomInvertibleMatrix(size int, random io.Reader) (*Matrix, *Matrix,
error) {
  for attempt := 0; attempt < 64; attempt++ {
    matrix, err := randomMatrix(size, size, random)
    if err != nil {
      return nil, nil, err
    }
    if inverse, ok := matrix.Invert(); ok {
```

```
      return matrix, inverse, nil
    }
  }
   return nil, nil, fmt.Errorf("mceliece: could not generate an invertible
matrix")
}

func randomMatrix(rows, cols int, random io.Reader) (*Matrix, error) {
  matrix := NewMatrix(rows, cols)
  bytesNeeded := (rows*cols + 7) / 8
  data := make([]byte, bytesNeeded)
  if _, err := io.ReadFull(random, data); err != nil {
    return nil, err
  }
  for i := 0; i < rows; i++ {
    for j := 0; j < cols; j++ {
      bit := i*cols + j
      if data[bit/8]&(0x80>>uint(bit%8)) != 0 {
        matrix.Set(i, j, true)
      }
    }
  }
  return matrix, nil
}

// randomErrorVector выбирает ровно weight различных позиций из n.
func randomErrorVector(n, weight int, random io.Reader) ([]uint64, error) {
  if weight > n {
    return nil, fmt.Errorf("mceliece: error weight %d exceeds block length %d",
weight, n)
  }
  vector := newVector(n)
  positions := make([]int, n)
  for i := range positions {
    positions[i] = i
  }
  for i := 0; i < weight; i++ {
    offset, err := randomBelow(random, n-i)
    if err != nil {
      return nil, err
    }
    j := i + offset
    positions[i], positions[j] = positions[j], positions[i]
    setVectorBit(vector, positions[i])
  }
  return vector, nil
}
```

Листинг А.4 – Адаптер к интерфейсу программы

```
package cipher

import (
  "fmt"

  "go-cipher/internal/crypto/mceliece"
)

type McEliece struct{}

func NewMcEliece() McEliece {
  return McEliece{}
}
```

```
func (McEliece) Name() string {
  return "McEliece"
}

// Открытый ключ — порождающая матрица G1 (k*n бит), закрытый — многочлен
// Гоппы, опорные элементы и S^{-1}. Размеры ключей на порядок больше
// модуля Рабина, поэтому панель показывает их placeholder'ом.
func (McEliece) KeyOptions() []KeyOption {
  // n = 1024, t = 38, k = 644 — параметры из методических указаний;
  // n = 256, t = 8, k = 192 — быстрый набор для демонстрации.
  return []KeyOption{
    {Label: "n=1024, t=38, k=644", Bits: 1024},
    {Label: "n=256, t=8, k=192", Bits: 256},
  }
}

func (McEliece) params(option KeyOption) (mceliece.Params, error) {
  switch option.Bits {
  case 1024:
    return mceliece.NewParams(10, 38)
  case 256:
    return mceliece.NewParams(8, 8)
  }
  return mceliece.Params{}, fmt.Errorf("%w: %d", mceliece.ErrUnsupported,
option.Bits)
}

func (m McEliece) GenerateKeys(option KeyOption) (KeyPair, error) {
  params, err := m.params(option)
  if err != nil {
    return KeyPair{}, err
  }
  public, private, err := mceliece.GenerateKeys(params)
  if err != nil {
    return KeyPair{}, err
  }
  return KeyPair{Public: public, Private: private}, nil
}

func (McEliece) Encrypt(pub, msg []byte) ([]byte, error) {
  key, err := mceliece.ParsePublicKey(pub)
  if err != nil {
    return nil, err
  }
  return key.Encrypt(msg)
}

func (McEliece) Decrypt(priv, cipherText []byte) ([]byte, error) {
  key, err := mceliece.ParsePrivateKey(priv)
  if err != nil {
    return nil, err
  }
  return key.Decrypt(cipherText)
}
```